

Lesson 5 — A Real Window: tkinter

One lesson: 2 hours 30 minutes · keep this sheet

Name _____

Class _____

Date _____

Cheat sheet

y points DOWN now

	turtle	tkinter
(0,0)	middle	top-left
y grows	up	down

```
y = y - 5      # moves UP
y = y + 5      # moves DOWN (gravity ADDS)
```

Turtle was the odd one out. This is the normal world.

Setup

```
window = tkinter.Tk()
canvas = tkinter.Canvas(window, width=W,
height=H,
    bg="#15181d", highlightthickness=0)
canvas.pack()
```

`highlightthickness=0` removes a 2-pixel border that silently breaks collision maths.

Rectangles are TWO CORNERS

```
canvas.create_rectangle(left, top, right,
bottom)
```

Not x, y, width, height. Write a helper:

```
def set_box(item, x, y, w, h):
    canvas.coords(item, x, y, x + w, y + h)
```

Create ONCE, then move

```
ball = canvas.create_oval(0,0,0,0,
fill="red") # before the loop
canvas.coords(ball, x, y, x+20, y+20)
# in the loop
```

Never `create_` inside the loop. The canvas keeps every shape forever and the game grinds to a halt.

Retained (tkinter)	Immediate (turtle, web)
system remembers shapes	you redraw everything
move what moved	clear and redraw

Removing

```
canvas.delete(fruit["item"]) # the
canvas's record
fruits.pop(i)                # AND yours
```

Two sets of books. Keep them in step.

The loop

```
window.after(16, game_loop)
```

Same as `ontimer` and `requestAnimationFrame`.

Input

```
def on_press(event):
    keys[event.keysym] = True
window.bind("<KeyPress>", on_press)
window.focus_force()
```

Falling

```
f["speed"] += GRAVITY * dt # accel changes
speed
f["y"]      += f["speed"]*dt # speed changes
position
```

Difficulty with a floor

```
delay = max(0.35, 1.2 - score * 0.004)
```

Design the limit at the same time as the slope.

Section A — Recall

A1. In tkinter, where is `(0, 0)` and which way does y grow? Write the line that makes something move *up*. [2 marks]

A2. What four numbers does `create_rectangle` take? [2 marks]

A3. Explain the difference between *retained mode* and *immediate mode*, and say which turtle and tkinter each are. [3 marks]

A4. `canvas.create_oval(...)` returns something. What is it? [2 marks]

A5. Why must you call `canvas.delete()` *as well as* removing the item from your own list? [3 marks]

Section B — Predict the output

B1. A fruit is at `y = 100` with speed 200, and `GRAVITY = 320`. After one frame of `dt = 0.016`, what is its speed and its y? Is it higher or lower on the screen? [3 marks]

B2. The game runs beautifully for about three seconds and then becomes jerky and then unusable. What is wrong, and roughly how many shapes exist after ten seconds at 60 fps? [4 marks]

```
def draw():
    canvas.create_oval(x - 20, 170, x + 20, 210, fill="red")
```

B3. What size and shape does this produce? The author wanted a basket 92 wide and 18 tall at `(100, 400)`. [3 marks]

```
canvas.create_rectangle(100, 400, 92, 18)
```

B4. `score` is 0, then 100, then 300. What is the spawn delay each time? What is the delay at a score of 10,000, and why? [4 marks]

```
delay = max(0.35, 1.2 - score * 0.004)
```

Section C — Find and fix the bug

C1. The fruit rises off the top of the screen instead of falling. One character is wrong. Which, and why? [3 marks]

```
fruit["y"] -= fruit["speed"] * dt
```

C2. Caught fruit disappears from the game — it stops counting for the score and stops colliding — but it stays visible on screen forever. What is missing? [4 marks]

C3. About half the fruit that reaches the bottom is never removed, and lives are lost at the wrong times. [4 marks]
Name the bug. You have seen it twice before — where?

```
for i in range(len(fruits)):
    if fruits[i]["y"] > HEIGHT:
        remove_fruit(i)
```

Section D — Build it

- ☐ **Checkpoint 1.** Get a tkinter window with a canvas, and draw a rectangle. Prove you understand the two-corners form by making it exactly 92×18 at a position you choose.
- ☐ **Checkpoint 2.** Write `set_box(item, x, y, w, h)`. Add the `game_loop` with `after(16, ...)` and the `dt` calculation with its clamp.
- ☐ **Checkpoint 3.** Add a basket you steer with the arrow keys, clamped inside the window. Create its canvas item **once**, outside the loop.
- ☐ **Checkpoint 4.** Make fruit spawn at the top at random x and fall. Keep them in a list of dictionaries, each holding its own canvas item id.
- ☐ **Checkpoint 5.** Catch the fruit with the AABB test. Add a score. Remove caught fruit from **both** your list and the canvas — looping backwards.
- ☐ **Checkpoint 6.** Add lives, a menu, a game-over screen, and a difficulty curve with a floor.

IF YOUR GAME SLOWS TO A CRAWL

You are creating canvas items inside the loop. Add `print(len(canvas.find_all()))` to `draw()` and watch the number climb into the

thousands. Create once, outside the loop; then only `coords()`.

Section E — Design challenge

NO RIGHT ANSWER

E1. Turtle made you redraw everything; tkinter remembers your shapes. For each of these, say which you would rather have and why: **a Snake game; a particle effect with 500 sparks; a chess board; a scrolling platform level.**

E2. You keep a Python list of fruit *and* the canvas keeps items. What goes wrong when the two disagree? Design a way to make it **impossible** for them to disagree.

E3. Your fruit speeds up as the score rises. At some point it will exceed human reaction time. How would you find that point **without guessing**? What would you change *instead* of speed?

E4. `after(16, game_loop)` asks for 60 fps. What happens if the window is dragged, or another program hogs the machine for half a second? What does the `dt` clamp protect you from — and what does the clamp *cost* you?

Stretch goals

1. Bad fruit you must avoid. Now the game is about choosing, not just moving.
2. A combo counter — and decide what breaks a combo.
3. Mouse control with `<Motion>`. Which feels better?

4. A real pause that stops the spawn timer too.
5. Draw your measured frames per second. Then add 500 fruit and watch what happens.